面向对象程序设计 / 超长记忆增强

Mandol
近重复记忆合并改进

目标系统理解、问题分析与初步方案

第 8/9 周初步汇报 | 2026. 10. 23

为什么想改进记忆合并

记录	内容
A	用户周五去上海参加会议。
B	用户周五去上海参加会议，周日返回。
C	如果请假获批，用户周五去上海。

想减少重复内容，但也要留下新增的信息和原始来源。

Mandol 在应用中负责什么

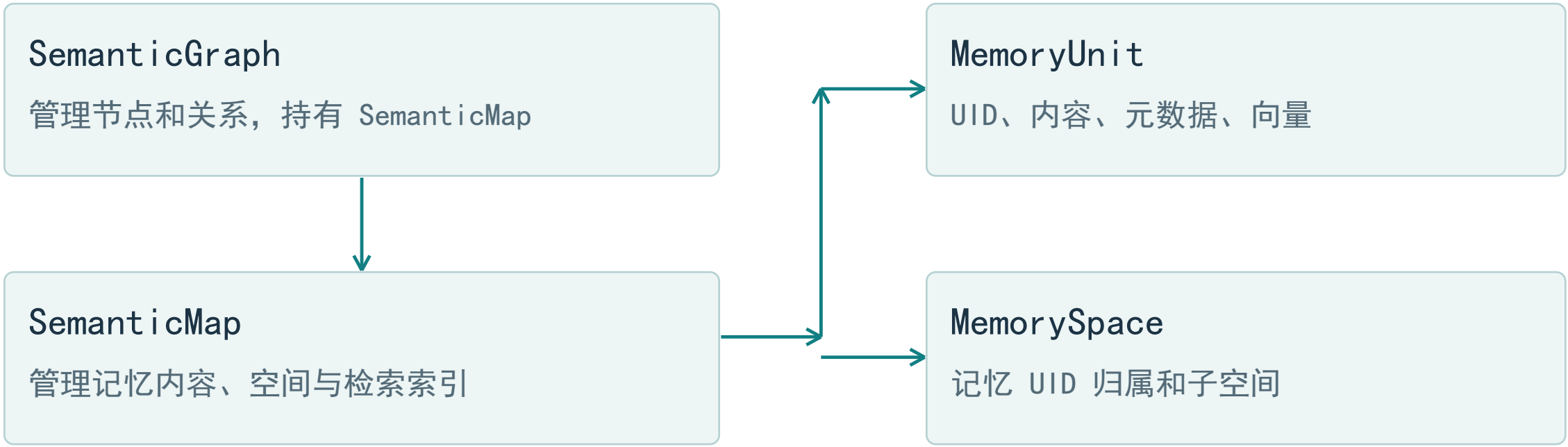
Mandol 负责保存和查找记忆，应用再把查到的内容交给模型生成回答。



这次主要考虑改进 L1 摘要生成之后的去重处理。

来源：README_PYPI.md；src/mandol/ 下的核心与构建模块

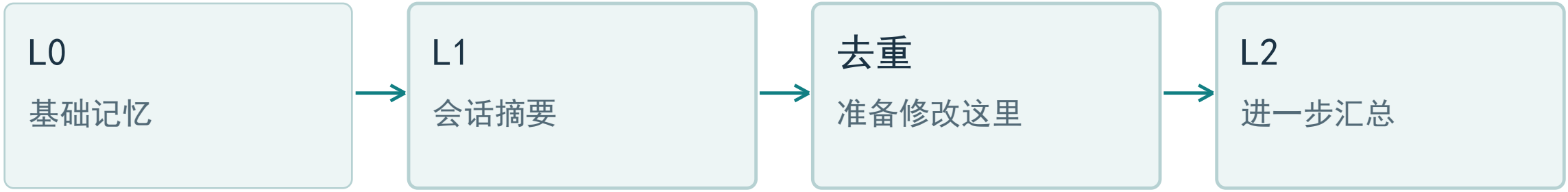
几个核心对象的分工



图中的箭头表示持有和管理关系。

来源：core/memory_unit.py、memory_space.py、semantic_map.py、semantic_graph.py

准备改在原流程的哪个位置



去重后的 L1 会继续生成 L2，之后由原来的流程写入系统。

保存一条 MemoryUnit，并不等于已经完成摘要和其他高阶记忆的构建。

来源：auto_builder/high_level_memory.py 的 _build_hierarchical()

现有 L1 去重怎样处理

先按摘要类型分组，将每条摘要的前 500 个字符交给模型判断。

```
merged_from = ["A", "B"]
kept_ids.add(merged_from[0])
```

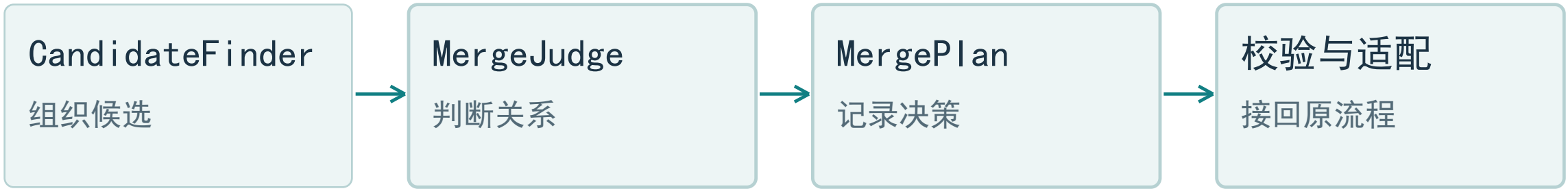
这段代码保留组内第一项的原文，没有采用模型返回的合并文本，也没有把其他条目的来源一起保存下来。

来源：auto_builder/hierarchical_builder.py 的 deduplicate_l1()，固定提交 6d7af4f

哪些内容可以合并，哪些先保留

情况	第一版准备怎样处理
同一事实的重复表达	保留一条，并汇总来源。
一条包含另一条的信息	先检查条件是否一致，再选择信息更全的一条。
否定、数字、时间或条件不同	保留两条，避免把不同事实混在一起。
无法确认或返回格式异常	先不合并，记录原因。

初步方案：先判断是否能合并



记录哪些条目被合并、留下哪条内容，以及来源、会话和判断理由。

第一版先选用已有文本，暂时不让模型重新写摘要。

目前先这样划分职责，具体接口还需要在实现时调整。

一个希望实现的合并例子

输入

A：周五去上海［来源 a］

B：周五去上海；周日返回［来源 b］

确认 B 包含 A 的信息后，希望得到

B：周五去上海；周日返回

来源：[a, b] 合并成员：[A, B]

当前进展与后续安排

已经完成

阅读主要源码，梳理对象和流程，找到 L1 去重的位置，整理初步方案。

接下来准备做

继续完善判断规则，把合并功能接入原流程；
再检查信息有没有丢失、来源是否完整，以及检索效果和处理成本。

当前仍是阶段性初稿，完整功能接入和系统验证尚未完成。

参考：课程大作业说明第1-3页；官方仓库 github.com/AgentCombo/Mandol；固定提交 6d7af4f